CloneScope - Phase 3 Learning Manual

Restricted Type-2 Clone Detection, Recursive Submission Analysis, and Explainable Reporting

Learning reference for Raian Huq (BSSE1708) | IIT, University of Dhaka

28 October 2026

Contents

Read this first: what this manual is, and what it is not **3**

1. The project in three levels of explanation **3**
1.1 One sentence for a non-technical person . . . 3
1.2 Thirty-second explanation for a classmate . . . 3
1.3 Technical explanation for a supervisor . . . 3
1.4 Input -> process -> output . . . 3

2. Build the mental model: characters, tokens, and regions **4**
2.1 Characters are not tokens . . . 4
2.2 `TokenType`, `Token`, and source position . . . 4
2.3 `CodeRegion` and the half-open interval rule . . . 4
2.4 Why compare function *bodies*? . . . 5

3. Type-1 vs restricted Type-2-only **5**
3.1 Exact Type-1 rule . . . 5
3.2 Restricted Type-2-only rule . . . 5
3.3 Why comments do not affect the match . . . 5

4. C++17 survival kit: every construct you need here **6**

5. Type-2 interface: header line by line **7**

6. Comparator: source-guided explanation **8**
6.1 Includes, signature, and input validation . . . 8
6.2 Equal-length requirement . . . 8
6.3 The two maps and renamed flag . . . 8
6.4 Visit aligned tokens . . . 9
6.5 The key step: consistent identifier names . . . 9
6.6 Reverse-map collision guard and insertion . . . 9
6.7 Non-identifiers and final return . . . 10

7. Trace the hash maps by hand **10**
7.1 Accepted example . . . 10
7.2 Forward conflict . . . 10
7.3 Reverse collision . . . 11
7.4 Changed literal and operator . . . 11

8. Multi-region discovery and complexity **11**
8.1 Implementation . . . 11
8.2 Why “reported candidates” are not independent student accusations . . . 11

9. Application integration: where the new module fits **12**

10. Recursive vs non-recursive scanning **12**
10.1 Exactly what y and n mean . . . 12
10.2 Actual C++ architecture, explained . . . 12
10.3 Relative paths preserve student identity in reports . . . 13
10.4 Why we need a scalable reporting stage later . . . 13

11. Explainable Type-2 reporting: code and tests **13**
11.1 Public reporting interface . . . 13
11.2 Defensive region checks in the reporter . . . 14
11.3 Source lines and why the -1 is essential . . . 14
11.4 What the report claims - and what it does not . . . 14
11.5 How `std::ostringstream` enables direct report testing . . . 15

12. Reproduce the tested experiments **15**
12.1 The 10 comparator/discovery checks . . . 15
12.2 The 5 reporting checks . . . 15
12.3 Controlled application results . . . 15
12.4 The six-file, three-student experiment . . . 15
12.5 A real debugging story: UTF-8 BOM . . . 16

13. PowerShell build, run, and verification commands **16**

14. Git history and disciplined engineering workflow **17**

15. Honest limitations and why they matter **17**

16. Complexity, performance, and the midterm plan **18**

17. Supervisor viva: question bank with short model answers **18**

18. Self-check exercises (answer after trying) **19**

19. Final checkpoint checklist **20**

Read this first: what this manual is, and what it is not

This manual explains the **implemented Phase 3 CloneScope prototype** in beginner-accessible language. It is a study companion for the source files, tests, Git history, and supervisor viva. It **does not** claim that CloneScope can detect every Type-2 clone or prove plagiarism. The actual implemented category is **restricted Type-2-only function-body candidates**: corresponding token types match; all non-identifier lexemes match; identifiers admit a consistent one-to-one renaming; and at least one identifier is actually renamed.

Verified evidence available at the end of Phase 3: ten Type-2 comparator/discovery checks passed; five Type-2 reporting checks passed; the six-file, three-student recursive fixture produced fifteen file pairs, one Type-1 candidate, and five Type-2-only candidates. These results concern controlled tests, **not precision/recall measured on authentic student submissions**. The Phase 3 branch was still local at the documentation checkpoint.

How to study: First understand the pipeline and matching rules. Next trace the real code line-by-line. Then reproduce the small fixtures and explain a result without looking at this manual. Last, rehearse the viva questions.

1. The project in three levels of explanation

1.1 One sentence for a non-technical person

CloneScope compares student programs and points instructors to **similar pieces of code**, explaining the locations and matching rules so that a human can review them.

1.2 Thirty-second explanation for a classmate

The current C++17 CLI scans `.c` and `.cpp` source files, optionally across nested student folders. It removes supported comments, tokenizes the code, finds supported function-body regions, then checks each pair for exact Type-1 body matches and consistently renamed Type-2-only body matches. It reports the source paths, original line ranges, classification, evidence rule, and a human-review warning.

1.3 Technical explanation for a supervisor

CloneScope is a **rule-based lexical/structural candidate discovery system**. Each analyzed source file is represented by a filesystem path, a vector of tokens (type, lexeme, line, column), and a vector of half-open token-index function-body regions. For each distinct source-file pair, the application compares region combinations. Type-1 requires exact corresponding `(type, lexeme)` equality. The Phase 3 Type-2-only comparator requires matching types and unchanged non-identifier lexemes while maintaining bidirectional identifier lexeme maps and observing at least one rename. Reporters are separated from detectors and validate region bounds before extracting source-line evidence.

1.4 Input -> process -> output

Stage	Current behavior	Why it matters
Input	Single path, or a folder with `y/n` recursion choice	Supports ordinary files and nested student folders
Preprocessing	`removeComments(sourceCode)`	Ignores supported comment differences for matching
Tokenization	`tokenize(cleanedCode)`	Produces comparable lexical units and positions
Region recognition	`findFunctionBodyRegions(tokens)`	Focuses comparison on selected function bodies
Pair generation	Each unordered source-file pair once	Avoids comparing A-B and B-A redundantly

Stage	Current behavior	Why it matters
Detection	Type-1 and restricted Type-2-only	Different evidence rules; no duplicate exact-match label
Reporting	Paths, source lines, rule, warning	Allows a reviewer to locate the evidence

The single-file route presently stops after preprocessing, tokenization, and region recognition. Cross-file comparisons run in directory mode.

2. Build the mental model: characters, tokens, and regions

2.1 Characters are not tokens

Consider this fragment:

```
return a + b;
```

A supported tokenizer can produce: `return` (Keyword), a (Identifier), + (Operator), b (Identifier), ; (Punctuation). It is **five** tokens regardless of spaces between the words. A comment such as `// add values` should not contribute comparison tokens after supported comment removal. The current implementation is not a complete language lexer; its behavior is limited to supported constructs.

2.2 `TokenType`, `Token`, and source position

The project's header defines the following conceptual representation (the fields are from the implemented tokenizer interface):

```
enum class TokenType
{
    Keyword, Identifier, Number, StringLiteral,
    CharLiteral, Operator, Punctuation, Unknown
};

struct Token
{
    TokenType type;
    std::string lexeme;
    std::size_t line;
    std::size_t column;
};
```

- `enum class TokenType` gives a type-safe set of categories. `TokenType::Identifier` names one value.
- `Token` groups four related facts into one object. `lexeme` is the exact recognized token spelling, such as `result` or +.
- `std::size_t` is an unsigned integer type commonly used for container sizes and indices.
- `line` and `column` are retained for human-readable evidence. The detector compares types and lexemes; it does **not** require matching source positions.
- `Unknown` is a category for unsupported non-whitespace characters, not evidence that the tool understands their semantics.

2.3 `CodeRegion` and the half-open interval rule

```
struct CodeRegion
{
    std::size_t beginTokenIndex;
    std::size_t endTokenIndex;
};
```

The start is **inclusive**; the end is **exclusive**. A region `{5, 9}` includes token indices `5, 6, 7, 8`, hence its length is `9 - 5 = 4`. The final included token is at `endTokenIndex - 1`. Empty regions (`begin >= end`) are rejected by the Type-2 comparator. Out-of-range ends are also rejected before indexing.

Reason for half-open regions: the length is an ordinary subtraction, adjacent regions can share a boundary without overlap, and loops naturally use `i < end`. This is common C++ container design.

2.4 Why compare function *bodies*?

In the current Phase 3 implementation, `findFunctionBodyRegions()` identifies selected body regions; signatures are outside those regions. Therefore:

```
int calculateTotal(int a, int b) { return a + b; }
int computeSum(int x, int y)   { return x + y; }
```

A body-level comparison considers `return a + b;` versus `return x + y;`. Names such as `calculateTotal` and `computeSum`, and parameter declarations in the signature, are not directly compared. **Do not say the full functions were compared.** This restricted, explicit scope is important in a viva.

3. Type-1 vs restricted Type-2-only

3.1 Exact Type-1 rule

For corresponding tokens in two valid body regions: same count, same types, same lexemes. `return a + b;` versus `return a + b;` qualifies; supported formatting or comment differences are irrelevant after preprocessing. `return a + b;` versus `return x + y;` does not qualify under Type-1 because identifier lexemes differ.

3.2 Restricted Type-2-only rule

All corresponding token **types** must match. Non-identifier **lexemes** must be exactly the same. Identifier lexemes may differ, but the substitution must be consistent and one-to-one through the entire compared region. At least one identifier must differ. An exact Type-1 match is intentionally **excluded** from Type-2-only output.

Body A	Body B	Current classification	Reason
`return a + b;`	`return a + b;`	Type-1	Same types and lexemes
`return a + b;`	`return x + y;`	Type-2-only	Consistent a->x, b->y
`return a + a;`	`return x + y;`	Neither	Same a would map to two names
`return a + b;`	`return x + x;`	Neither	Two names would map to x
`return a + b;`	`return x - y;`	Neither	Operator lexeme changed
`return a + 1;`	`return x + 2;`	Neither	Literal lexeme changed

Terminology caution: Some textbook Type-2 definitions permit more variation than our initial identifier-only matcher. Our report's phrase *Type-2-only* means it is not also reported as exact Type-1 under the currently implemented rules; it does not imply full Type-2 language coverage.

3.3 Why comments do not affect the match

Source content first flows through `removeComments()`. Supported `//` and `/* ... */` comments are removed while ordinary literals and physical line breaks are preserved in supported cases.

As a result, two identical bodies with different comments can match as Type-1, and consistently renamed bodies with different comments can match as Type-2-only.

Important for future stylometry: A comment is discarded for matching, but original comments and indentation may be interesting supplementary style measurements. Therefore, future style analysis must retain or separately access **original source text**. Stylometry is **not** implemented in Phase 3.

4. C++17 survival kit: every construct you need here

This section is for reading the actual code, not memorizing abstract jargon.

Syntax	Read it aloud	Role in CloneScope
`#pragma once`	Include this header only once per translation unit	Avoids duplicate header definitions
`#include <vector>`	Import vector declarations	Stores tokens, regions, matches
`#include "type2_clone.hpp"`	Include local interface	Connects `.cpp` to declarations
`std::vector<Token>`	Growable sequence of `Token` objects	Analyzed token stream
`const std::vector<Token>&`	Read-only reference to vector	Avoids copying potentially large token arrays
`const CodeRegion&`	Read-only reference to one region	Passes token bounds efficiently
`std::size_t`	Size/index integer type	Array indexing and lengths
`bool`	`true` or `false`	Candidate decision
`std::unordered_map<K,V>`	Hash-based key -> value mapping	Identifier substitutions
`auto existing = map.find(key);`	Deduce iterator type	Locate existing identifier mapping
`map.end()`	One-past-end iterator	Test whether lookup found anything
`existing->second`	Value field at iterator's pair	Inspect mapped identifier
`map.emplace(key,value)`	Insert a new key/value pair	Record a correspondence
`&&`, `||`, `!`	AND, OR, NOT	Combine and invert logical tests
`return false;`	Reject current region pair immediately	Fast fail on mismatch
`for (const CodeRegion& r : regions)`	Visit every region without copying	Enumerate candidate regions
`++i`	Increment index	Advance token loop
`.push_back(x)`	Append an element	Save matching region pair
`std::ostream&`	Reference to an output destination	Testable reporter abstraction
`std::ostringstream`	String-backed output stream	Test report text in memory
`std::filesystem::path`	Structured filesystem path	Identify source files

4.1 Why `const` and `&` appear together

In `const std::vector<Token> &firstTokens`, `&` denotes a **reference** rather than a copied vector; `const` prohibits changing the referenced vector through that parameter. Large vectors can be passed efficiently and safely. It is not pointer dereferencing here.

4.2 Why `std::` appears before names

`std` is the C++ standard-library namespace; `::` selects a name inside it. `std::vector` and `std::string` are different standard-library types. The code uses an alias `namespace fs = std::filesystem;` in the application file, so `fs::path` abbreviates `std::filesystem::path`.

4.3 What `auto` does and does not do

`auto existing = forwardMap.find(first.lexeme);` asks the compiler to infer an iterator type from the expression's result. It does **not** make C++ dynamically typed. The iterator's concrete type is determined at compile time.

4.4 What an iterator represents

`unordered_map::find(key)` returns an iterator to a matching key/value pair, or `end()` if no entry exists. A map stores entries as pairs; `iterator->first` accesses the key and `iterator->second` accesses the associated value. Comparing the iterator with `end()` must happen **before** dereferencing it.

4.5 Why && matters for bounds

Logical AND short-circuits: if its left operand is false, the right side is not evaluated. In our multi-region smoke test, `multiMatches.size() == 1 && multiMatches[0]...` prevents indexing `[0]` when the collection does not contain exactly one element.

5. Type-2 interface: header line by line

This is the Phase 3 header interface as supplied during development:

```
#pragma once
#include <vector>
#include "code_region.hpp"

struct Type2BodyMatch
{
    CodeRegion firstRegion;
    CodeRegion secondRegion;
};

bool areType2BodyCandidates(
    const std::vector<Token> &firstTokens,
    const CodeRegion &firstRegion,
    const std::vector<Token> &secondTokens,
    const CodeRegion &secondRegion);

std::vector<Type2BodyMatch> findType2BodyMatches(
    const std::vector<Token> &firstTokens,
    const std::vector<CodeRegion> &firstRegions,
    const std::vector<Token> &secondTokens,
    const std::vector<CodeRegion> &secondRegions);
```

- `#pragma once`: the header's contents are processed once per translation unit.
- `#include <vector>`: needed for declarations involving `std::vector`.
- `#include "code_region.hpp"`: provides `CodeRegion` and, through its own include, the token declarations used by these function signatures.
- `struct Type2BodyMatch`: one finding consists of **two regions**, one from each analyzed file. This struct does not contain a plagiarism verdict or similarity score.
- `bool areType2BodyCandidates(...)`: compare **one** specified region pair; return accept/reject.
- `std::vector<Type2BodyMatch> findType2BodyMatches(...)`: compare **all** supplied region combinations; return all qualifying pairs.
- A trailing semicolon after a **function declaration** is required; the implementation in `.cpp` defines the function body instead.

Supervisor question: Why separate two functions? **Answer:** One encapsulates the matching rule; the second enumerates candidate region pairs and reuses that rule, avoiding duplicate logic.

6. Comparator: source-guided explanation

The comparator excerpts below are drawn from the implemented `src/type2_clone.cpp` shared and tested during Phase 3. Each block is followed by the purpose of every meaningful statement or control decision.

6.1 Includes, signature, and input validation

```
#include "type2_clone.hpp"
#include <unordered_map>

bool areType2BodyCandidates(
    const std::vector<Token> &firstTokens,
    const CodeRegion &firstRegion,
    const std::vector<Token> &secondTokens,
    const CodeRegion &secondRegion)
{
    if (firstRegion.beginTokenIndex >= firstRegion.endTokenIndex ||
        secondRegion.beginTokenIndex >= secondRegion.endTokenIndex ||
        firstRegion.endTokenIndex > firstTokens.size() ||
        secondRegion.endTokenIndex > secondTokens.size())
    {
        return false;
    }
```

`#include <unordered_map>` makes hash maps available. The four parameters tell the comparator **which two token arrays** and **which two ranges** to examine. The `if` condition rejects an empty/reversed range (`begin >= end`) or an end past the vector's size. `||` means any invalid condition is enough to reject. This is defensive validation **before** indexing the vectors. Since indices are unsigned, checking `begin >= end` first also prevents subtracting an invalid reversed range later.

6.2 Equal-length requirement

```
    std::size_t firstLength =
        firstRegion.endTokenIndex - firstRegion.beginTokenIndex;

    std::size_t secondLength =
        secondRegion.endTokenIndex - secondRegion.beginTokenIndex;

    if (firstLength != secondLength)
    {
        return false;
    }
```

Length is measured in **tokens**, not characters or lines. A changed identifier spelling can have a different number of characters, yet still occupy exactly one token. Equal token length permits aligned comparison: token `i` in region A with token `i` in region B.

6.3 The two maps and renamed flag

```
    std::unordered_map<std::string, std::string> forwardMap;
    std::unordered_map<std::string, std::string> reverseMap;
    bool hasRenamedIdentifier = false;
```

`forwardMap` records first-body name -> second-body name; `reverseMap` records second-body name -> first-body name. Together they enforce a **bijection over the identifier lexemes encountered**, rather than a many-to-one mapping. `hasRenamedIdentifier` starts false and becomes true when at least one corresponding identifier differs, ensuring exact Type-1 bodies are not duplicated in Type-2-only results.

6.4 Visit aligned tokens

```
for (std::size_t i = 0; i < firstLength; ++i)
{
    const Token &first =
        firstTokens[firstRegion.beginTokenIndex + i];

    const Token &second =
        secondTokens[secondRegion.beginTokenIndex + i];

    if (first.type != second.type)
    {
        return false;
    }
```

- The loop starts at zero and runs while `i < firstLength`, visiting each token position exactly once.
- The expression `beginTokenIndex + i` maps a **relative position** within a region to its **absolute token-vector index**.
- `const Token&` avoids copying a token and prohibits changing it through that reference.
- A token-type mismatch immediately rejects a candidate. An identifier cannot be paired with a keyword or punctuation under this algorithm.

6.5 The key step: consistent identifier names

```
    if (first.type == TokenType::Identifier)
    {
        if (first.lexeme != second.lexeme)
        {
            hasRenamedIdentifier = true;
        }

        auto existing = forwardMap.find(first.lexeme);
        if (existing != forwardMap.end())
        {
            if (existing->second != second.lexeme)
            {
                return false;
            }
        }
```

- The outer `if` means the special renaming rule applies only when the aligned token is an identifier (the earlier type check ensures the second is also an identifier).
- A differing name marks this pair as having a rename. It is **not yet proof of a valid match**; subsequent checks could reject the pair.
- `forwardMap.find(first.lexeme)` looks for a previously seen name from the first body.
- If found, the same source name must map to the same target name on every occurrence. Otherwise `return false` catches a **forward inconsistency**.

6.6 Reverse-map collision guard and insertion

```
        else
        {
            auto reverseExisting = reverseMap.find(second.lexeme);

            if (reverseExisting != reverseMap.end())
            {
                return false;
            }

            forwardMap.emplace(first.lexeme, second.lexeme);
            reverseMap.emplace(second.lexeme, first.lexeme);
```

```
            }
        }
```

The `else` runs only for an unseen source identifier. If the proposed target identifier is already in the reverse map, another first-body identifier would be using the same target, so the comparator rejects the collision. Otherwise, `emplace` adds entries in **both directions**. The two maps remain synchronized for accepted steps.

Why can the code reject on any `reverseExisting` rather than compare its mapped value? Because this branch is reached only when the first-body name has **not** appeared in `forwardMap`; any already-claimed target name would therefore represent a conflicting pairing.

6.7 Non-identifiers and final return

```
        else if (first.lexeme != second.lexeme)
        {
            return false;
        }
    }

    return hasRenamedIdentifier;
}
```

The `else if` handles keywords, punctuation, operators, numbers, and literals (and any other non-identifier token categories): their **lexeme text must be identical**. When the loop completes without mismatch, the function returns the renamed flag. An exact body returns false here (it belongs to Type-1); a consistently renamed body returns true.

Code-review caveat: This rule compares identifier *lexemes*, not the scope/binding identity of variables. It does not resolve shadowed variables or semantics as a compiler does.

7. Trace the hash maps by hand

7.1 Accepted example

Body A:

```
int sum = a + b;
return sum;
```

Body B:

```
int total = x + y;
return total;
```

Encounter	First identifier	Second identifier	Forward map after step	Reverse-map result
1	`sum`	`total`	`{sum:total}`	`total` claimed by `sum`
2	a	x	add {a:x}	x claimed by a
3	b	y	add {b:y}	y claimed by b
4	`sum`	`total`	existing value agrees	no new insertion

At least one rename occurred, no conflicting mapping exists, and non-identifier lexemes agree. **Result: accepted as restricted Type-2-only.**

7.2 Forward conflict

`return a + a;` versus `return x + y;` begins with `a -> x`. At the next occurrence, the same first-body identifier a would need to map to y. `existing->second != second.lexeme` becomes true. **Result: reject.**

7.3 Reverse collision

`return a + b;` versus `return x + x;` first stores `a -> x`. Then `b` is unseen but `x` has already been claimed in `reverseMap`. **Result: reject.**

7.4 Changed literal and operator

`return a + 1;` versus `return x + 2;` is rejected when comparing 1 to 2. Similarly, `return a + b;` versus `return x - y;` is rejected on `+` versus `-`. The algorithm does not "understand" that expressions might be equivalent; it enforces precise lexical rules.

8. Multi-region discovery and complexity

8.1 Implementation

```
std::vector<Type2BodyMatch> findType2BodyMatches(
    const std::vector<Token> &firstTokens,
    const std::vector<CodeRegion> &firstRegions,
    const std::vector<Token> &secondTokens,
    const std::vector<CodeRegion> &secondRegions)
{
    std::vector<Type2BodyMatch> matches;

    for (const CodeRegion &firstRegion : firstRegions)
    {
        for (const CodeRegion &secondRegion : secondRegions)
        {
            if (areType2BodyCandidates(
                    firstTokens, firstRegion,
                    secondTokens, secondRegion))
            {
                matches.push_back(
                    Type2BodyMatch{firstRegion, secondRegion});
            }
        }
    }

    return matches;
}
```

The empty `matches` vector is a place to collect results. A **nested range-based for loop** visits every combination of recognized first-file and second-file regions. The matching function is called for each combination. `Type2BodyMatch{firstRegion, secondRegion}` creates a small record holding the two boundaries; `push_back` appends it. `return matches` returns the collection, which can be empty without error.

If the two files contain `r` and `s` recognized regions, the discovery function attempts `r * s` region comparisons. For similarly sized regions of `L` tokens, the comparator is **typically O(L)** time under average hash-map operations, so discovery is roughly **O(r * s * L)** for such a simplified case; total work varies with actual region lengths and early rejections. Per-comparison maps need space proportional to the distinct identifiers encountered.

8.2 Why "reported candidates" are not independent student accusations

Two programs can contain several matching short function bodies. Each valid region pair is a reported **candidate**, not a count of students found guilty. A high candidate count is not by itself a measured similarity percentage or plagiarism likelihood. Common teaching templates and standard algorithms may match legitimately.

9. Application integration: where the new module fits

The existing `src/application.cpp` processes selected sources and retains `AnalyzedSourceFile { path, tokens, functionRegions }`. Inside the **file-pair** loop, it already calls Type-1 discovery/reporting. Phase 3 calls Type-2 discovery **independently**, regardless of whether Type-1 matches were found:

```
std::vector<Type2BodyMatch> type2Matches =
    findType2BodyMatches(
        first.tokens,
        first.functionRegions,
        second.tokens,
        second.functionRegions);

cout << "Type-2 body-level candidates: "
     << type2Matches.size() << '\n';

if (!type2Matches.empty())
{
    printType2BodyMatches(
        cout,
        firstRelativePath,
        first.tokens,
        secondRelativePath,
        second.tokens,
        type2Matches);
}
```

`std::vector<Type2BodyMatch>` holds the returned matches. `.size()` is the number of recorded region pairs. `.empty()` determines whether the report should print. The filename variables are **relative paths**, not merely `filename()`; this matters when all three students upload a file named `fibonacci.c`.

Viva point: The application coordinates data movement; `type2_clone` performs matching; `type2_report` performs presentation. This separation makes focused tests possible.

10. Recursive vs non-recursive scanning

10.1 Exactly what `y` and `n` mean

Given:

```
submissions/
  student1/fibonacci.c
  student1/prime.c
  student2/fibonacci.c
  student2/prime.c
```

Choosing `n` examines only supported files **directly in `submissions/`**; these four nested files would not be processed. Choosing `y` traverses nested student folders as well. Both modes later filter regular files for exactly lowercase `.c` and `.cpp` extensions.

10.2 Actual C++ architecture, explained

```
std::vector<fs::directory_entry> entries;
std::string scanChoice;

cout << "Scan subfolders recursively? (y/n): ";
getline(cin, scanChoice);

if (scanChoice == "y" || scanChoice == "Y")
{
```

```
    for (const auto &entry :
         fs::recursive_directory_iterator(sourcePath))
    {
        entries.push_back(entry);
    }
}
else
{
    for (const auto &entry :
         fs::directory_iterator(sourcePath))
    {
        entries.push_back(entry);
    }
}

for (const auto &entry : entries)
{
    // Existing file filtering and processing runs here.
}
```

- `fs` is an alias for `std::filesystem` defined in `application.cpp`.
- `std::vector<fs::directory_entry>` holds discovered directory-entry metadata.
- `getline(cin, scanChoice)` captures a response from the console.
- `"y" || "Y"` enables recursive traversal; the current `else` also catches invalid inputs, so anything other than y/Y behaves as non-recursive.
- `recursive_directory_iterator` can traverse nested directories; `directory_iterator` visits its only direct entries.
- `entries.push_back(entry)` accumulates discovered entries for the existing processing loop.
- **Important previous bug we fixed:** iterating `for (const auto& entry : entries)` while `entries` was initially empty discovered no files. The input loop must iterate the actual filesystem iterator; only the **processing** loop should iterate `entries`.

10.3 Relative paths preserve student identity in reports

```
const std::string firstRelativePath =
    first.path.lexically_relative(sourcePath).generic_string();
```

`lexically_relative(sourcePath)` removes the chosen root path prefix lexically. `generic_string()` produces a slash-separated string suitable for reports, such as `student1/fibonacci.c`. This makes a match between identically named files in different folders interpretable. It does **not** automatically infer a structured student ID or assignment ID for statistics.

10.4 Why we need a scalable reporting stage later

For `n` analyzed source files, the number of **distinct unordered file pairs** is `n * (n - 1) / 2`. Thus six files yield 15 pairs; sixty would yield 1,770. The current implementation prints verbose preprocessed source text plus pairwise results. A future statistical/CSV/JSON stage can group evidence and reduce console overload. Those features were **not implemented in Phase 3**.

11. Explainable Type-2 reporting: code and tests

11.1 Public reporting interface

```
void printType2BodyMatches(
    std::ostream &output,
    const std::string &firstFileName,
    const std::vector<Token> &firstTokens,
    const std::string &secondFileName,
    const std::vector<Token> &secondTokens,
    const std::vector<Type2BodyMatch> &matches);
```

Passing `std::ostream&` instead of hardcoding `std::cout` makes the function usable with terminal streams and a test `std::ostringstream`. The reporter receives **already discovered matches**; it does not perform Type-2 detection itself.

11.2 Defensive region checks in the reporter

```
for (const Type2BodyMatch &match : matches)
{
    const CodeRegion &first = match.firstRegion;
    const CodeRegion &second = match.secondRegion;

    if (first.beginTokenIndex >= first.endTokenIndex ||
        first.endTokenIndex > firstTokens.size() ||
        second.beginTokenIndex >= second.endTokenIndex ||
        second.endTokenIndex > secondTokens.size())
    {
        output << "Warning: skipped invalid match boundaries.\n";
        continue;
    }
```

- `for (... : matches)` visits each provided result record.
- `first` and `second` are references to the region fields inside that record.
- Each region must be nonempty and in bounds.
- `continue` **skips only this invalid record** and proceeds to the next one; it does not terminate the whole report.
- The test `tests/type2_report_smoke.cpp` independently passed an invalid region and confirmed a warning, zero valid reported candidates, and no misleading `Candidate #1` line.

11.3 Source lines and why the -1 is essential

```
const Token &firstStart =
    firstTokens[first.beginTokenIndex];

const Token &firstEnd =
    firstTokens[first.endTokenIndex - 1];
```

The start token identifies the first line for the region. Because the end index is **exclusive**, the last included token lives at `endTokenIndex - 1`. The reporter uses `.line` on these tokens, not line numbers invented by the detector. It does the analogous operation for the second region. Checking bounds *first* ensures these indexed expressions are safe for the handled cases.

11.4 What the report claims - and what it does not

The reporter prints candidate numbering, relative file paths, line ranges, a generic description of consistent one-to-one renaming, `Type-2-only body candidate`, and the exact human-review caution: **“Human review required; similarity is not proof of plagiarism.”** It does **not** currently print a table of identifier correspondences or a similarity percentage.

An observed report from the three-student experiment:

```
Comparing files: student1/fibonacci.c vs student3/fibonacci.c
Type-1 body-level candidates: 0
Type-2 body-level candidates: 1
CloneScope Type-2 body-level candidate report

Candidate #1
student1/fibonacci.c: lines 4 to 19
student3/fibonacci.c: lines 4 to 19
Evidence: consistent one-to-one identifier renaming with
matching non-identifier token lexemes.
Classification: Type-2-only body candidate.
Human review required; similarity is not proof of plagiarism.
```

```
Reported candidates: 1
```

Note: The evidence line above is wrapped for page layout; in the actual CLI it is printed as one line.

11.5 How `std::ostringstream` enables direct report testing

```
std::ostringstream output;
printType2BodyMatches(
    output, "student1/demo.c", firstTokens,
    "student2/demo.c", secondTokens, matches);
const std::string report = output.str();

bool pathsCorrect =
    report.find("student1/demo.c") != std::string::npos;
```

The report writes into a string buffer. `.str()` retrieves the text. `.find(...)` searches it; `std::string::npos` is the special "not found" result. Thus `!= npos` means the expected path exists in the report. Other assertions check classification, human-review warning, count, and invalid boundary handling.

12. Reproduce the tested experiments

12.1 The 10 comparator/discovery checks

`tests/type2_clone_smoke.cpp` passed: consistent renaming; inconsistent forward mapping rejection; reverse collision rejection; exact Type-1 exclusion; operator mismatch rejection; literal mismatch rejection; valid region accepted; empty region rejected; out-of-bounds region rejected; multi-region discovery.

12.2 The 5 reporting checks

`tests/type2_report_smoke.cpp` passed: relative path text; classification text; human-review warning; reported count; invalid-region rejection.

Correct interpretation: These are fifteen **specific local checks**, not a formally measured system-wide accuracy score.

12.3 Controlled application results

Dataset / selection	Files	File pairs	Type-1	Type-2-only
`tests/fixtures/type1`, choose n	4	6	6	0
`tests/fixtures/type2`, choose n	2	1	0	1
`tests/fixtures/type2/three_students`, choose y	6	15	1	5

The separate `factorial` fixture exercises renamed variables in factorial/main function bodies. The `classroom_six` fixture interleaves different assignments among six students. These supplementary local runs were reported successful during development; the full numerical table above is based on results explicitly shown in the conversation.

12.4 The six-file, three-student experiment

```
tests/fixtures/type2/three_students/
  student1/fibonacci.c
  student1/prime.c
  student2/fibonacci.c
  student2/prime.c
```

```
student3/fibonacci.c
student3/prime.c
```

All files were recognized (one supported function body per file). Six files produce 15 file pairs, and the outputs included exactly these intended match relationships:

Pair of files	Outcome
`student1/fibonacci.c` vs `student2/fibonacci.c`	Type-1
`student1/fibonacci.c` vs `student3/fibonacci.c`	Type-2-only
`student2/fibonacci.c` vs `student3/fibonacci.c`	Type-2-only
`student1/prime.c` vs `student2/prime.c`	Type-2-only
`student1/prime.c` vs `student3/prime.c`	Type-2-only
`student2/prime.c` vs `student3/prime.c`	Type-2-only

The remaining nine file pairs yielded zero candidates in this controlled fixture. This demonstrates correct **expected fixture behavior**, not proof of authorship or general detection accuracy.

12.5 A real debugging story: UTF-8 BOM

An earlier `.cpp` fixture was saved using Windows PowerShell's `Set-Content -Encoding utf8`, which in Windows PowerShell 5.1 can add a UTF-8 BOM. The preprocessed console output displayed stray characters and the tokenizer reported 24 tokens per file. Saving the fixture as UTF-8 **without** BOM removed the stray prefix and the token count became 21 per file, while the intended Type-2-only match remained. **Lesson:** input-encoding handling is a limitation worth testing and documenting; cleaning a fixture is not the same as implementing general BOM support.

13. PowerShell build, run, and verification commands

Run from the repository root `C:\Users\raian\Developer\SPL_01_BSSE1708` using a C++17-capable g++.

Full application build:

```
g++ -std=c++17 -Wall -Wextra src/main.cpp src/application.cpp
 ↪  src/preprocessor.cpp src/tokenizer.cpp src/code_region.cpp
 ↪  src/function_region.cpp src/type1_clone.cpp src/type1_report.cpp
 ↪  src/type2_clone.cpp src/type2_report.cpp -o clonescope.exe
```

Application launch:

```
.\clonescope.exe
```

For the recursive student-folder test, enter `tests/fixtures/type2/three_students`, then answer y. For non-recursive Type-1 regression, enter `tests/fixtures/type1`, then answer n.

Type-2 comparator/discovery tests:

```
g++ -std=c++17 -Wall -Wextra tests/type2_clone_smoke.cpp src/type2_clone.cpp
 ↪  src/tokenizer.cpp -o type2_clone_smoke.exe
.\type2_clone_smoke.exe
```

Type-2 report tests:

```
g++ -std=c++17 -Wall -Wextra tests/type2_report_smoke.cpp src/type2_report.cpp
 ↪  src/tokenizer.cpp -o type2_report_smoke.exe
.\type2_report_smoke.exe
```

Git review, without publishing:

```
git status -sb
git diff --check
git log -3 --oneline
```

A successful compiler return with no diagnostics proves the named sources compiled and linked for that command; a successful test run proves only the assertions that were actually executed.

14. Git history and disciplined engineering workflow

Phase 3 was implemented on local branch `feat/type2-clone-detection`. Three confirmed local commits were recorded:

Commit	Recorded purpose
e4b156d	Type-2 identifier mapping and region matching
9a658e9	Type-2 reporting and recursive source discovery
2c6e033	Reporting and invalid-region regression tests

Commit versus push versus merge: `git commit` records a snapshot locally; `git push` publishes committed branch history to the remote; opening a Pull Request requests review; merging integrates the branch's changes into `main`. The user's established workflow is: learn, implement, verify, test, make meaningful local commits, document, final review, **then** publish/PR/merge. At this manual's checkpoint, Phase 3 was not yet published or merged.

README and technical notes describe **what is already implemented**. This learning manual explains **how to understand and defend it**. Do not mix future feature promises into implemented-results tables.

15. Honest limitations and why they matter

Limitation	Consequence	Appropriate later response
Restricted body recognition	Some real C/C++ functions may be missed	Expand supported patterns with regression tests
Identifier-only Type-2	Literal/operator changes rejected	Broaden Type-2 definition only when deliberately implemented
No symbol binding/scopes	Shadowing can confuse lexeme-based mappings	Document scope and later consider binding-aware improvements
Only `.c` / `.cpp` lowercase	Other extensions skipped in folder mode	Extend filtering deliberately
Verbose console output	Large runs become hard to review	Introduce concise summaries and CSV/JSON
No assignment-aware filtering	Compares unrelated tasks and same-student files	Add optional comparison organization later
No detailed rename table	Report describes generic evidence, not exact maps	Expose concrete identifier correspondence later
UTF-8 BOM not robust	Prefixed bytes can be tokenized unexpectedly	Add input-encoding handling and fixtures
No automated CI	Tests currently run locally	Configure repeatable CI later if in scope
No large benchmark yet	60-80-file scaling and accuracy unverified	Benchmark labeled 30-40-student data before midterm

Human-review boundary: Two students might independently write the same short Fibonacci or prime-number code. CloneScope's result must be discussed as a *similarity candidate*. Do not equate identical bodies, renamed identifiers, or candidate counts with misconduct.

16. Complexity, performance, and the midterm plan

For `n` analyzed source files, file pairs = `n(n-1)/2`; for 60 files the mathematical count is 1,770 distinct pairs. If two files contain `r` and `s` recognized function bodies, that pair may involve `r*s` region-pair checks. Individual aligned comparisons are typically linear in their token length under average hash-map performance. The actual end-to-end runtime also depends on scanning, file I/O, preprocessing, tokenization, reporting, and region distribution. **No runtime has been verified for 60 files in Phase 3.**

The agreed pre-midterm development priority is: initial stylometric and statistical analysis; better-supported Type-1/Type-2 coverage; concise CSV/JSON output; a labeled 30-40-student / 60-80-file classroom-style evaluation; and an integrated demonstration. Actual student submissions require appropriate permission and anonymization. Restricted Type-3 and Type-4 remain **after-midterm work**.

17. Supervisor viva: question bank with short model answers

Q1. What exactly does your Phase 3 implementation do?
A. It finds supported function-body token pairs with consistent one-to-one identifier renaming and identical corresponding non-identifier lexemes, excluding exact Type-1 bodies from the Type-2-only output.

Q2. Why call it "restricted Type-2-only"?
A. The matcher currently permits only identifier renaming. It does not cover all conventional Type-2 variations, such as some literal changes, and it excludes exact Type-1 matches by design.

Q3. Does your tool detect plagiarism?
A. No. It reports explainable code-similarity candidates requiring human interpretation.

Q4. Why are there two unordered maps?
A. The forward map ensures repeated source identifiers map consistently; the reverse map prevents two different source identifiers from mapping to one target identifier.

Q5. What does `hasRenamedIdentifier` mean?
A. At least one aligned identifier lexeme differed. Only after all other checks pass does that flag allow Type-2-only classification.

Q6. Why compare token types first?
A. The correspondence rules assume aligned categories. A keyword paired with an identifier is immediately rejected.

Q7. Why compare non-identifier lexemes exactly?
A. That is the deliberately restricted Phase 3 rule, preventing operator or literal variations from being silently accepted.

Q8. Why do you validate boundaries?
A. Empty or out-of-range regions must be rejected before indexing a token vector, preventing invalid element access in the supported checks.

Q9. Why is `endTokenIndex` exclusive?
A. It makes region length `end - begin`, and the last included token is `end - 1`.

Q10. Why use const references for vectors?
A. Avoid potentially expensive copies while preventing mutation through the function parameter.

Q11. What is the role of `findType2BodyMatches()`?
A. It enumerates all cross-file region pairs and calls the single-region comparison function, collecting valid matches.

Q12. Why not put reporting in the comparator?
A. Separation of concerns makes detection reusable and report formatting independently testable.

Q13. What is the benefit of `std::ostream&` in the reporter?
A. The same function can write to terminal output or a string buffer used in regression tests.

Q14. What does `std::string::npos` represent?
A. It is the result of an unsuccessful string search; testing `find(...) != npos` checks presence of expected report text.

Q15. What does `y` mean in directory scanning?
A. Traverse nested subdirectories using `recursive_directory_iterator`.

Q16. What does `n` mean?
A. Visit only directly contained entries using `directory_iterator`.

Q17. Why preserve relative paths?
A. Multiple students may submit `fibonacci.c`; a report must distinguish `student1/fibonacci.c` from `student3/fibonacci.c`.

Q18. Why don't you compare file signatures or full semantics?
A. Current region recognition and comparison are intentionally restricted to supported function bodies. It is not a full parser or semantic equivalence engine.

Q19. What did your three-student experiment show?
A. Six nested `.c` files, fifteen file pairs, one expected Type-1 candidate, and five expected Type-2-only candidates, with relative-path reporting and nine other pairs rejected.

Q20. What do the 10+5 smoke checks establish?
A. Correct behavior for fifteen selected test assertions. They are not measured population-level accuracy, precision, recall, or proof of plagiarism.

Q21. Can CloneScope analyze 30 folders with two programs each?
A. Its design is not hardcoded to six files and can enumerate 60 discovered files, giving 1,770 file pairs. But we have not yet verified time, memory, or accuracy at that scale.

Q22. Why does the tokenizer show different counts when BOM is present?
A. The observed fixture encoded prefix was treated unexpectedly. Removing the BOM fixed the fixture, but general BOM support is still an implementation limitation.

Q23. Why do Type-1 and Type-2-only counts not overlap in our current design?
A. Type-2-only returns true only if at least one identifier differs, so an exact token match is left to Type-1.

Q24. Why will CSV/JSON help?
A. They can make many comparisons filterable and machine-readable; they do not by themselves establish accuracy or create a visualization.

Q25. How will you address false positives?
A. Document common boilerplate and short-code effects, build labeled tests, analyze patterns statistically, and require human review before any academic conclusion.

18. Self-check exercises (answer after trying)

Exercise A: For `return a + a;` vs `return p + q;`, is it Type-2-only?
Answer: No. Repeated a cannot map consistently to both p and q.

Exercise B: For `return a + b;` vs `return p + p;`, is it Type-2-only?
Answer: No. Reverse mapping would require both a and b to map to p.

Exercise C: Why do `{2,7}` and `{7,10}` not overlap under half-open ranges?
Answer: The first includes indices 2-6, and the second includes 7-9.

Exercise D: A directory contains 12 recognized files. How many unordered file pairs?
Answer: `12*11/2 = 66`.

Exercise E: A report says zero Type-2-only candidates. Can we infer the code is unrelated?
Answer: No. It only says no supported region pair satisfied this restricted rule.

Exercise F: Two source files have three and two recognized bodies. How many Type-2 region-pair candidates are checked before early rejection?
Answer: Up to `3*2 = 6` region-pair comparisons.

Exercise G: A `.C` file is inside a recursive folder. Will it currently be selected?
Answer: No, the current filter checks lowercase `.c` or `.cpp` exactly.

Exercise H: Why is `std::ostringstream` a strong design for reporting tests?
Answer: It captures full report text in memory so tests can check exact content and warnings without reading terminal screenshots.

19. Final checkpoint checklist

- I can explain characters, tokens, token types, and half-open regions.
- I can trace `forwardMap` and `reverseMap` without looking at code.
- I can distinguish Type-1, restricted Type-2-only, and rejected pairs.
- I understand the comparison code's validation and early-return decisions.
- I can explain why nested loops enumerate region pairs and file pairs.
- I can explain `y/n` scanning and student-relative reporting paths.
- I can run both smoke-test executables and the three-student fixture.
- I can cite the **actual** verified counts without inventing accuracy percentages.
- I can defend human-review safeguards and current limitations.
- I know Phase 3 was not published or merged at the documentation checkpoint.

One-sentence viva conclusion: "CloneScope Phase 3 adds an explainable, identifier-renaming-based Type-2-only function-body candidate detector to an existing Type-1 token pipeline, with recursive student-folder analysis and human-review-oriented evidence reporting."